

Міністерство освіти і науки України
Національний університет «Запорізька політехніка»

Кафедра програмних засобів

ЗВІТ

Дисципліна «Технології розподілених систем та паралельних обчислень»

Робота №1

Тема «Основи програмування та засоби синхронізації в багатозадачній
операційній системі»

Виконав варіант 19

Студент КНТ-122

Онищенко О.А.

Прийняли

Викладач

Дьячук Т.С.

МЕТА

Метою роботи є ознайомитися із застосуванням різних методів синхронізації при створенні багатопоточних застосунків.

ЗАВДАННЯ

Завдання на роботу:

1. Відкомпілювати й запустити запропонований приклад простий консольної багатопоточної комп'ютерної гри «Війна програмних потоків».
2. Розібрати текст програми ThreadWar.cpp. Знайти й підкреслити в тексті програми функції WinAPI для роботи з об'єктами синхронізації. Пояснити їх застосування.

ВИКОНАННЯ

Текст програми ThreadWar.cpp

```
// Проста комп'ютерна гра Thread War
// Використовуйте клавіші "уліво" і "вправо", щоб переміщати пушку
// клавіша "пробіл" робить постріл,
// Якщо 30 ворогів підуть із екрана не знищеними, ви програли
// Очки даються за кожного вбитого супротивника

#include <windows.h>
#include <process.h>
#include <stdlib.h>
#include <time.h>
#include <stdio.h>

// Об'єкти синхронізації
HANDLE screenlock;    // зміною екрана займається тільки один потік
HANDLE bulletsem;     // можна вистрілити тільки три рази підряд
HANDLE startevt;      // гра починається з натисканням клавіші "уліво"
або "вправо"
HANDLE conin, conout; // дескриптори консолі
HANDLE mainthread;    // основний потік main
CRITICAL_SECTION gameover;
```

```

CONSOLE_SCREEN_BUFFER_INFO info; // інформація про консоль
// кількість влучень і промахів
long hit = 0;
long miss = 0;
long delayfactor = 7; // фактор затримки для ворогів

// Constants for fixed screen size
const int SCREEN_WIDTH = 80;
const int SCREEN_HEIGHT = 25;

// Function to fix console size to prevent scrolling
void SetupConsole()
{
    SMALL_RECT windowSize = {0, 0, SCREEN_WIDTH - 1, SCREEN_HEIGHT -
1};
    COORD bufferSize = {SCREEN_WIDTH, SCREEN_HEIGHT};

    // Set window size first to ensure it fits the buffer changes
    SetConsoleWindowInfo(conout, TRUE, &windowSize);
    SetConsoleScreenBufferSize(conout, bufferSize);
    // Set window size again to be sure
    SetConsoleWindowInfo(conout, TRUE, &windowSize);

    // Disable cursor visibility
    CONSOLE_CURSOR_INFO cursorInfo;
    GetConsoleCursorInfo(conout, &cursorInfo);
    cursorInfo.bVisible = FALSE;
    SetConsoleCursorInfo(conout, &cursorInfo);
}

// Створення випадкового числа від n0 до n1
int random(int n0, int n1)
{
    if (n0 == 0 && n1 == 1)
        return rand() % 2; // спеціальний випадок
    return rand() % (n1 - n0) + n0;
}

// Очищення екрана консолі
void cls()
{
    COORD org = {0, 0};
    DWORD res;
    FillConsoleOutputCharacter(conout, ' ', SCREEN_WIDTH *
SCREEN_HEIGHT, org, &res);
}

// вивести на екран символ в позицію x і y
void writeat(int x, int y, char c)
{
    // Check bounds to prevent writing off-screen (which causes
scroll/glitch)
    if (x < 0 || x >= SCREEN_WIDTH || y < 0 || y >= SCREEN_HEIGHT)
        return;

    // Блокувати вивід на екран за допомогою м'ютекса
    WaitForSingleObject(screenlock, INFINITE);
    COORD pos = {(SHORT)x, (SHORT)y};

```

```

        DWORD res;
        WriteConsoleOutputCharacterA(conout, &c, 1, pos, &res);
        ReleaseMutex(screenlock);
    }

    // Одержати натискання на клавішу (лічильник повторень в ct)
    int getakey(int &ct)
    {
        INPUT_RECORD input;
        DWORD res;
        while (1)
        {
            ReadConsoleInput(conin, &input, 1, &res);

            // ігнорувати інші події
            if (input.EventType != KEY_EVENT)
                continue;

            // ігнорувати події відпускання клавіш
            // нас цікавлять тільки натискання
            if (!input.Event.KeyEvent.bKeyDown)
                continue;
            ct = input.Event.KeyEvent.wRepeatCount;
            return input.Event.KeyEvent.wVirtualKeyCode;
        }
    }

    // Обробка комбінацій ^C, ^Break, і т.і.
    BOOL WINAPI ctrl(DWORD type)
    {
        exit(0);
        return TRUE;
    }

    // Визначити символ в заданій позиції екрана
    int getat(int x, int y)
    {
        char c;
        DWORD res;
        COORD org = {(SHORT)x, (SHORT)y};

        // Блокувати доступ до консолі доти, поки процедура не буде
        виконана
        WaitForSingleObject(screenlock, INFINITE);
        ReadConsoleOutputCharacterA(conout, &c, 1, org, &res);
        ReleaseMutex(screenlock); // unlock
        return c;
    }

    // Відобразити очки в заголовку вікна й перевірити умову завершення гри
    void score(void)
    {
        char s[128];
        sprintf_s(s, "Thread War! Hit: %d Miss : %d", hit, miss);
        SetConsoleTitleA(s);
        if (miss >= 30)
        {
            EnterCriticalSection(&gameover);

```

```

        SuspendThread(mainthread); // призупинити головний потік
        MessageBoxA(NULL, "Game Over!", "Thread War", MB_OK |
MB_SETFOREGROUND);
        exit(0); // не виходить із критичної секції
    }
    if ((hit + miss) % 20 == 0 && delayfactor > 0) // Added check > 0
        InterlockedDecrement(&delayfactor);
}

char badchar[] = "-\\|/";

// це потік супротивника
void badguy(void *_y)
{
    int y = (int)(uintptr_t)_y; // Cast safe for x64
    int dir;
    int x;

    // FIX: Right side spawn should be Width - 1, not Width (which is
out of bounds)
    x = y % 2 ? 0 : SCREEN_WIDTH - 1;

    // установити напрямок залежно від початкової позиції
    dir = x ? -1 : 1;

    // поки супротивник перебуває в межах екрана
    while ((dir == 1 && x < SCREEN_WIDTH) || (dir == -1 && x >= 0))
    {
        int dly;
        BOOL hitme = FALSE;
        // перевірка на влучення (куля?)
        if (getat(x, y) == '*')
            hitme = TRUE;

        // вивід символу на екран
        writeat(x, y, badchar[x % 4]);
        // ще одна перевірка на влучення
        if (getat(x, y) == '*')
            hitme = TRUE;

        // перевірка на влучення через невеликі проміжки часу
        if (delayfactor < 3)
            dly = 3;
        else
            dly = delayfactor + 3;

        for (int i = 0; i < dly; i++)
        {
            Sleep(40);
            if (getat(x, y) == '*')
            {
                hitme = TRUE;
                break;
            }
        }
        writeat(x, y, ' ');
        // ще одна перевірка на влучення
        if (getat(x, y) == '*')

```

```

        hitme = TRUE;
    if (hitme)
    {
        // у супротивника влучили!
        MessageBeep(-1);
        InterlockedIncrement(&hit);
        score();
        _endthread();
    }
    x += dir;
}
// супротивник утік!
InterlockedIncrement(&miss);
score();
}

// цей потік займається створенням потоків супротивників
void badguys(void *)
{
    // чекаємо сигналу до початку гри протягом 15 секунд
    WaitForSingleObject(startevt, 15000);
    while (1)
    {
        if (random(0, 100) < (hit + miss) / 25 + 20)
        {
            // згодом імовірність збільшується
            // Spawn enemies at Y=1 to Y=10
            _beginthread(badguy, 0, (void *) (uintptr_t) (random(1,
10))));
            Sleep(1000); // щосекунди
        }
    }
}

// Це потік кулі
void bullet(void *_xy_)
{
    COORD xy = *(COORD *) _xy_;
    if (getat(xy.X, xy.Y) == '*')
        return;

    if (WaitForSingleObject(bulletsem, 0) == WAIT_TIMEOUT)
        return;

    // FIX: Logic was while(-xy.Y) which loops forever in place.
    // Changed to decrement Y to move bullet UP.
    while (xy.Y > 0)
    {
        writeat(xy.X, xy.Y, '*'); // відобразити кулю
        Sleep(100);
        writeat(xy.X, xy.Y, ' '); // стерти кулю
        xy.Y--; // Move bullet UP
    }
    // постріл зроблений - додати 1 до семафора
    ReleaseSemaphore(bulletsem, 1, NULL);
}

// Основна програма

```

```

void main()
{
    HANDLE me;
    conin = GetStdHandle(STD_INPUT_HANDLE);
    conout = GetStdHandle(STD_OUTPUT_HANDLE);
    SetConsoleCtrlHandler(ctrl, TRUE);
    SetConsoleMode(conin, ENABLE_WINDOW_INPUT);

    // FIX: Set fixed console size
    SetupConsole();

    me = GetCurrentThread();

    DuplicateHandle(GetCurrentProcess(), me, GetCurrentProcess(),
&mainthread, 0, FALSE, DUPLICATE_SAME_ACCESS);
    startevt = CreateEvent(NULL, TRUE, FALSE, NULL);
    screenlock = CreateMutex(NULL, FALSE, NULL);
    InitializeCriticalSection(&gameover);
    bulletsem = CreateSemaphore(NULL, 3, 3, NULL);
    GetConsoleScreenBufferInfo(conout, &info);

    score();
    srand((unsigned)time(NULL));
    cls();

    // установка початкової позиції пушки
    int y = SCREEN_HEIGHT - 2; // Keep it slightly above absolute
bottom
    int x = SCREEN_WIDTH / 2;

    _beginthread(badguys, 0, NULL);

    // основний цикл гри
    while (1)
    {
        int c, ct;
        writeat(x, y, '|'); // намалювати пушку
        c = getkey(ct);      // одержати символ
        switch (c)
        {
            case VK_SPACE: // вогонь!
            {
                // Using static here is risky in threads, but kept for
simplicity of original code structure.
                // Ideally, pass a copy of struct.
                static COORD xy;
                xy.X = x;
                xy.Y = y - 1; // Fire from above the gun

                _beginthread(bullet, 0, (void *)&xy);
                Sleep(100);
                break;
            }
            case VK_LEFT: // команда "уліво!"
                SetEvent(startevt);
                writeat(x, y, ' ');
                while (ct--)
                    if (x > 0)

```

```

        x--;
        break;
    case VK_RIGHT: // команда "вправо!"
        SetEvent(startevt);
        writeat(x, y, ' ');

        // FIX: "while (ct)" was an infinite loop because ct was
never changed.
        // Changed to "while (ct--)"
        while (ct--)
            if (x < SCREEN_WIDTH - 1)
                x++;
        break;
    }
}
}

```

Вигляд запущеної програми наведено на рисунку нижче:

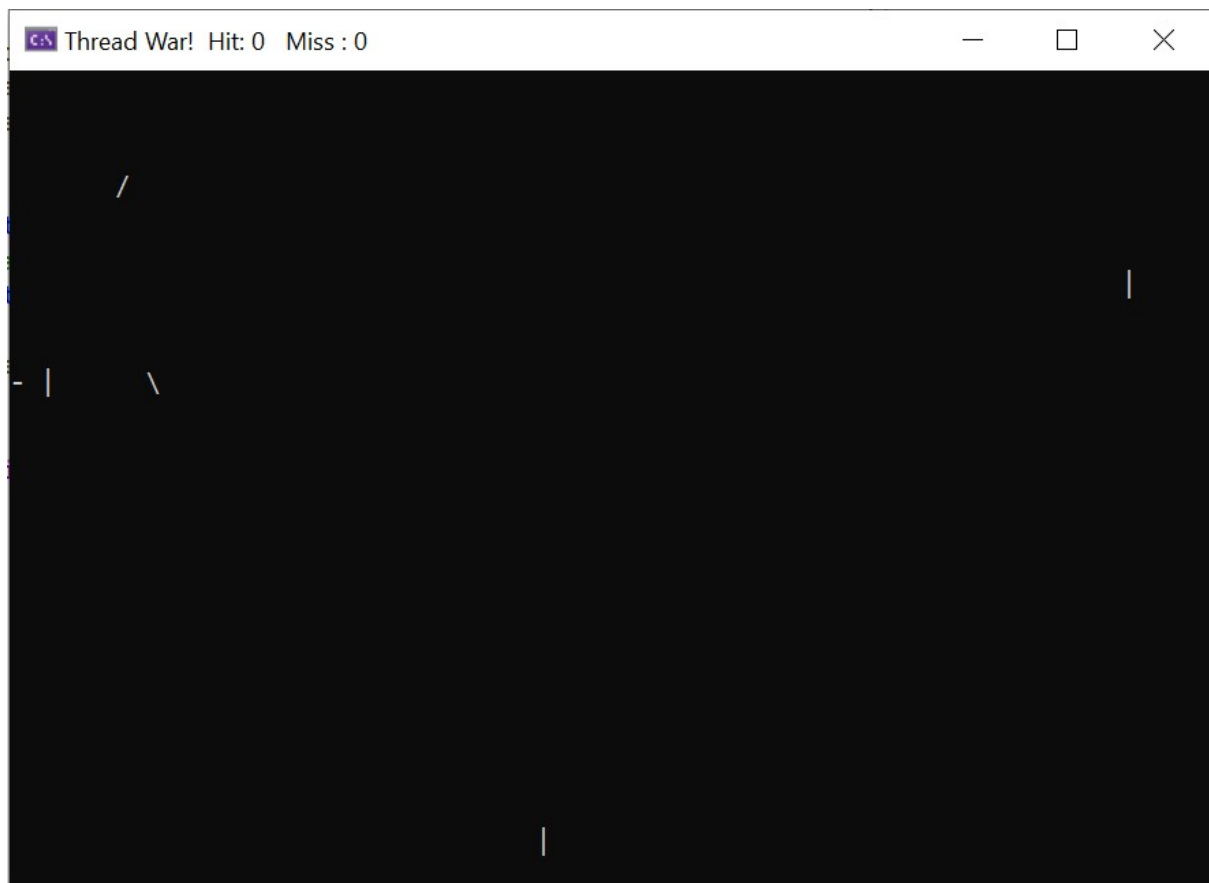


Рисунок 1.1 — Вигляд програми

Схема взаємодії потоків

Схему взаємодії потоків наведено на рисунку нижче:

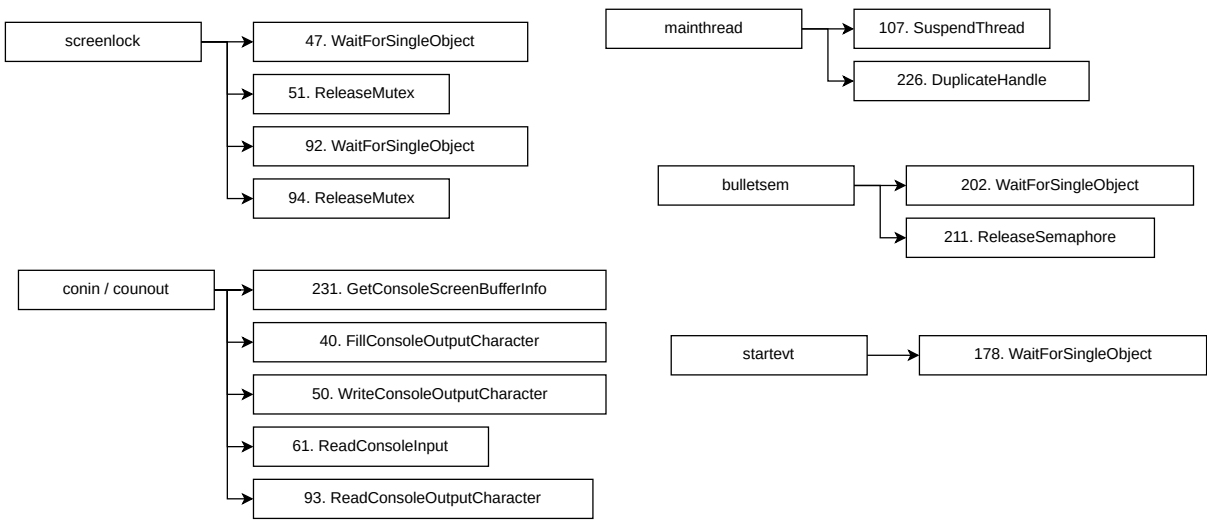


Рисунок 1.1 — Схема взаємодії потоків

Опис функцій Windows API, що зустрічаються в програмі

Описи функцій, знайдених у програмі, наведено нижче на таблиці:

Таблиця 1.1 — Функції Windows API, знайдені у програмі

Рядок	Функція	Призначення	Параметри	Значення, що повертаються
47, 92	WaitForSingleObject	Очікує поки вказаний об'єкт перейде в сигнальний стан або мине інтервал очікування.	Screenlock, INFINITE	При успішному виконанні функції повернуте значення вказує на подію, що стала причиною повернення результату. Може приймати такі значення: WAIT_ABANDONED, WAIT_OBJECT_0, WAIT_TIMEOUT, WAIT_FAILED
178			Startevt, 15000	
202			Bulletsem, 0	
51, 94	ReleaseMutex	Звільняє право власності на вказаний об'єкт mutex.	Screenlock	При успішному виконанні повертає ненульове значення. При неуспішному виконанні повертає нульове значення
106	EnterCriticalSection	Чекає на власника вказаного об'єкта критичної секції. Функція повертається, коли потоку, що	&gameover	Не повертає значення, викликає EXCEPTION_POSSIBLE_DEADLOCK

Рядок	Функція	Призначення	Параметри	Значення, що повертаються
		викликає, було надано право власності.		
107	SuspendThread	Призупиняє вказаний потік	Mainthread	При успішному виконанні функція повертає значення попереднього лічильника зупиненого потоку. Інакше — (DWORD) - 1
187	_beginthread	Створює потік	Badguy, 0, random(1,10)	При успішному виконанні повертає дескриптор новоствореного потоку.
243			Badguy, 0	
258			Bullet, 0, &xy	
211	ReleaseSemaphore	Збільшує кількість вказаного семафорного об'єкта на задану величину	Bulletsem, 1	При успішному виконанні повертає нуль.
206, 208	Writeat	Записує задану кількість байтів, починаючи з заданого зсуву.	xy.X, xy.Y	Повертає одне з наступних значень: - S_OK; - E_FAIL; - E_PENDING; - STG_E_ACCESSDENIED; - STG_E_WRITEFAULT; - STG_E_MEDIUMFULL.
223	GetCurrentThread	Отримує псевдо-дескриптор для потоку, що викликає.	-	Не повертає значення.
226	DuplicateHandle	Дублює дескриптор об'єкта	GetCurrentProcess, GetCurrentProcess, mainthread, 0	При успішному виконанні повертає нуль.
227	CreateEvent	Створює або відкриває іменованний або безіменний об'єкт події.	True, False	При успішному виконанні повертає дескриптор об'єкта події.
228	CreateMutex	Створює або відкриває іменованний або безіменний mutex-об'єкт.	False	При успішному виконанні повертає дескриптор створеного mutex-об'єкта.
229	InitializeCriticalSection	Ініціалізує об'єкт критичного розділу.	&gameover	Не повертає значення.
230	CreateSemaphore	Створює або відкриває іменованний або безіменний семафорний об'єкт.	3, 3	При успішному виконанні повертає дескриптор об'єкту семафору.

ПИТАННЯ

Поняття процесу і потоку. Стани процесу.

Процес — екземпляр програми, що виконується.

Потік — послідовність інструкцій, яка виконується всередині процесу.

Стани процесу наведено на таблиці нижче:

Таблиця 2.1 — Стани процесу

Назва	Опис
Створений (<i>New / Created</i>)	Процес створено системою.
Готовий (<i>Ready</i>)	Процес завантажено у пам'ять.
Виконуваний (<i>Running</i>)	Інструкції процесу активно виконуються на процесорі.
Заблокований (<i>Blocked / Waiting</i>)	Процес не може бути виконаний, бо чекає на подію (наприклад, завершення введення / виведення, звільнення ресурсу)
Завершений (<i>Terminated / Exit</i>)	Процес припинив роботу та звільнив ресурси

Багатозадачність. Багатопроцесорність.

Багатозадачність — здатність операційної системи одночасно обробляти кілька процесів.

Багатопроцесорність — використання двох чи більше ядер процесора в одній системі для одночасного виконання різних частин програми (або декількох програм).

Моделі взаємодії процесів. Конкуренція.

Моделі взаємодії процесів — механізм, який дозволяє процесам обмінюватися даними та синхронізувати свої дії.

Конкуренція — здатність системи обробляти кілька завдань одночасно.